CPU SCHEDULING ALGORITHMS PERFORMANCE
FINAL PROJECT REPORT

Course: Data Analysis (L-31)

Operating Systems

Professors:

Prof. M. Giacobbe,

Prof. M. Fazio

Authors:

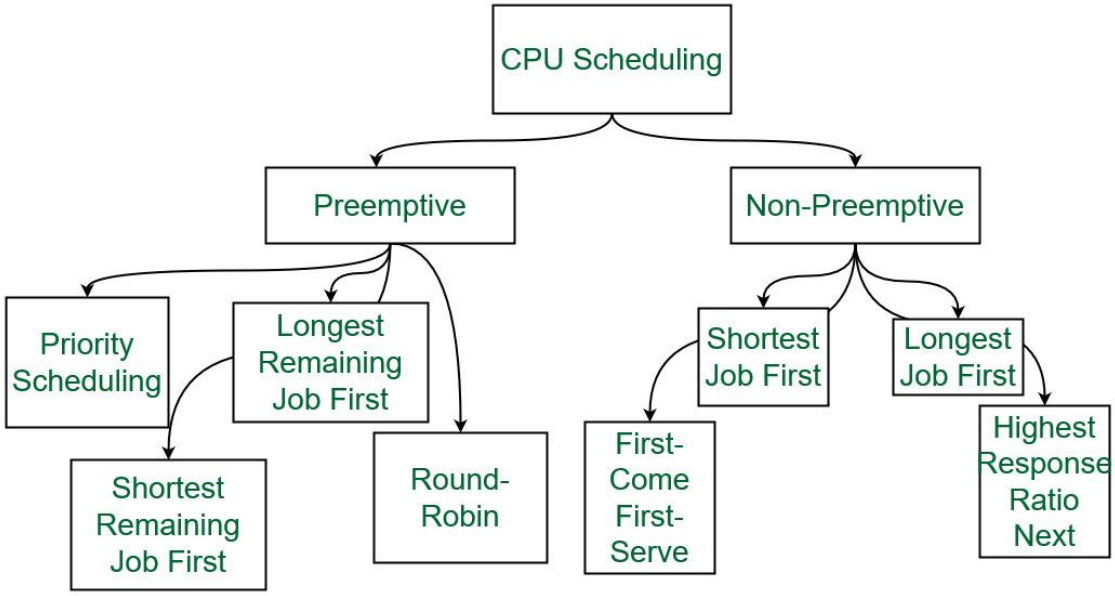
Mehdi Talebikhatir (ID: 558948)

Benyamin Baharizadeh (ID: 560587)

UniMe

1548

Project Overview and Introduction



CPU scheduling is a fundamental component of operating system design that directly impacts system performance, resource utilization, and overall user experience. As modern systems juggle numerous simultaneous processes, the strategy used to allocate CPU time becomes critical. This project develops a comprehensive simulation and analysis framework to evaluate the performance characteristics of five essential CPU scheduling algorithms.

Through the implementation of these algorithms from scratch in Python, this project provides a hands-on simulation to visualize how different strategies handle identical workloads. The resulting tool successfully provides empirical evidence comparing algorithm efficiency across various workload scenarios through interactive visual analytics, Gantt charts, and statistical metric comparisons.

Project Objectives

The primary objectives of this project are divided into implementation, analysis, and educational goals:

Implementation: Develop accurate implementations of five CPU scheduling algorithms with both preemptive and non-preemptive variants within a modular, extensible codebase.

Analysis: Quantify performance metrics (waiting time, turnaround time, response time, throughput), conduct comparative benchmarking, and analyze phenomena such as the convoy effect and starvation.

Educational: Deepen the understanding of process scheduling theory, operating system internals, and data visualization.

Implemented Scheduling Algorithms

The simulation fully implements five fundamental scheduling algorithms. Below is an in-depth look at their mechanics and trade-offs:

First-Come-First-Served (FCFS)
A non-preemptive algorithm that schedules processes strictly in the order they arrive in the ready queue. While simple to implement using a standard FIFO queue, it is highly prone to the 'Convoy Effect,' where short processes get stuck waiting behind a single long CPU-bound process, drastically increasing average waiting times.

Shortest Job First (SJF)
A non-preemptive algorithm that selects the waiting process with the smallest execution (burst) time. SJF is mathematically proven to provide the minimum average waiting time for a given set of processes. However, it can lead to the starvation of longer processes if short processes continually arrive.

Round Robin (RR)
A preemptive algorithm designed specifically for time-sharing systems. Each process receives a small, fixed unit of CPU time (a time quantum, set to q=2 in this simulation). If a process does not finish within this quantum, it is preempted and placed at the back of

the ready queue. This ensures fair CPU distribution and excellent response times, though setting the quantum too small increases context-switching overhead.

Priority Scheduling
A non-preemptive algorithm where the CPU is allocated to the process with the highest priority (represented by the lowest integer value). In the event of a tie, FCFS is used to break it. This method can suffer from priority inversion and starvation, where low-priority processes may wait indefinitely.

Shortest Remaining Time First (SRTF)
The preemptive counterpart to SJF. The scheduler evaluates the remaining time of the currently running process against the burst time of newly arriving processes. If a new process requires less time than the currently running process has remaining, the current process is preempted. This yields exceptionally low average waiting times but requires frequent context switching.

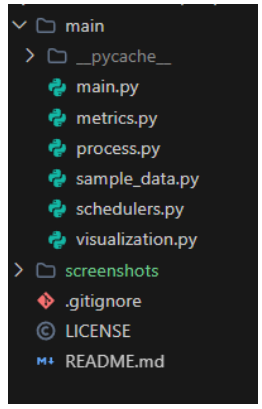
Workload Test Scenarios

To thoroughly evaluate the algorithms, the simulation includes six carefully designed workload scenarios:

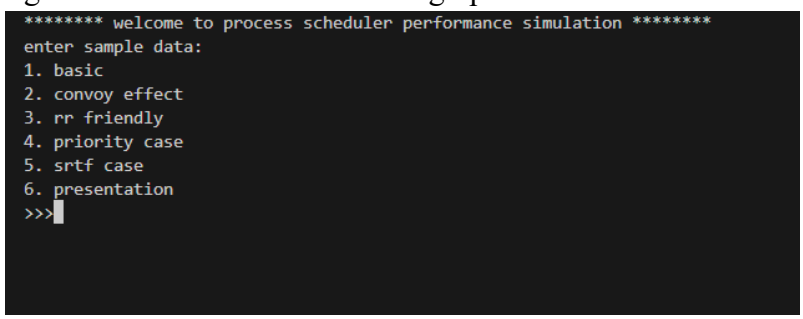
Basic Scenario: A baseline case with moderate process arrival times and burst durations to test general functionality.
Convoy Effect: Features a massive CPU-bound process arriving early, followed by several very short processes. This explicitly demonstrates the weaknesses of FCFS.
RR Friendly: Simultaneous arrivals with varying burst times, designed to favor time-sharing strategies and show how Round Robin prevents early arriving long jobs from monopolizing the CPU.
Priority Case: A diverse set of priority values to evaluate priority inversion risks and strict priority-based scheduling mechanics.
SRTF Case: Staggered, continuous arrivals specifically optimized to trigger preemptions and showcase the efficiency of shortest-remaining-time logic.
Presentation: A complex, mixed workload serving as an ultimate stress test for a comprehensive algorithm comparison.

Technical Architecture & System Usage

The project is written entirely in Python 3.x, utilizing a modular structure:



Data Modeling: Python's 'dataclasses' are used to cleanly define Process attributes (PID, arrival, burst, priority).
Visualization: Matplotlib and NumPy power the visual output, calculating statistics and generating accurate Gantt charts and bar graphs.



Interface: An interactive Command-Line Interface (CLI) allows users to dynamically select a scenario. The simulation then processes the workload through all five algorithms simultaneously and compiles the visual output.

Analytical Metrics & Visual Results

The simulation calculates and visualizes several critical performance metrics:

Average Waiting Time: The total time a process spends waiting in the ready queue. SRTF consistently minimized this metric across test scenarios.

Turnaround Time: The total time taken from a process's arrival to its completion. (Turnaround = Waiting Time + Burst Time).

Response Time: The time from arrival until the first time the process is scheduled on the CPU. Round Robin excelled here, confirming its suitability for interactive systems.

Throughput: The number of processes completed per unit of time.

To satisfy the requirement for comprehensive evaluation, the simulation outputs both graphical and numerical representations of the results for all implemented algorithms.

Per-Process Detailed Results
Scenario: Selected Scenario

FCFS - Process Results

PID	Arrival	Burst	Start	Completion	Waiting	Turnaround	Response
P1	0	15	0	15	0	15	0
P2	1	3	15	18	14	17	14
P3	2	6	18	24	16	22	16
P4	3	1	24	25	21	22	21
P5	4	8	25	33	21	29	21
P6	5	4	33	37	28	32	28

SJF - Process Results

PID	Arrival	Burst	Start	Completion	Waiting	Turnaround	Response
P1	0	15	0	15	0	15	0
P2	1	3	16	19	15	18	15
P3	2	6	23	29	21	27	21
P4	3	1	15	16	12	13	12
P5	4	8	29	37	25	33	25
P6	5	4	19	23	14	18	14

RR (q=2) - Process Results

PID	Arrival	Burst	Start	Completion	Waiting	Turnaround	Response
P1	0	15	0	37	22	37	0
P2	1	3	2	12	8	11	1
P3	2	6	4	24	16	22	2
P4	3	1	8	9	5	6	5
P5	4	8	9	32	20	28	5
P6	5	4	12	22	13	17	7

Priority - Process Results

PID	Arrival	Burst	Start	Completion	Waiting	Turnaround	Response
P1	0	15	0	15	0	15	0
P2	1	3	15	18	14	17	14
P3	2	6	23	29	21	27	21
P4	3	1	18	19	15	16	15
P5	4	8	29	37	25	33	25
P6	5	4	19	23	14	18	14

SRTF - Process Results

PID	Arrival	Burst	Start	Completion	Waiting	Turnaround	Response
P1	0	15	0	37	22	37	0
P2	1	3	1	4	0	3	0
P3	2	6	9	15	7	13	7
P4	3	1	4	5	1	2	1
P5	4	8	15	23	11	19	11
P6	5	4	5	9	0	4	0

Figure 1: Numerical Representation of Results.

A detailed per-process data table calculating the precise metrics (Waiting, Turnaround, and Response times) on a per-process basis for every implemented algorithm.

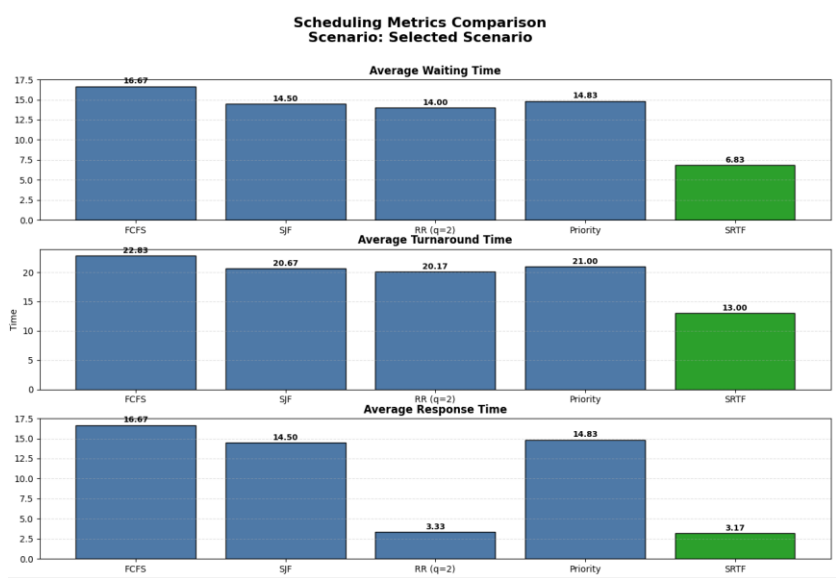


Figure 2: Graphical Comparison of Evaluating Metrics.

Side-by-side metric comparison bar charts visualizing Average Waiting Time, Average Turnaround Time, and Average Response Time, instantly highlighting the optimal algorithm for the workload.

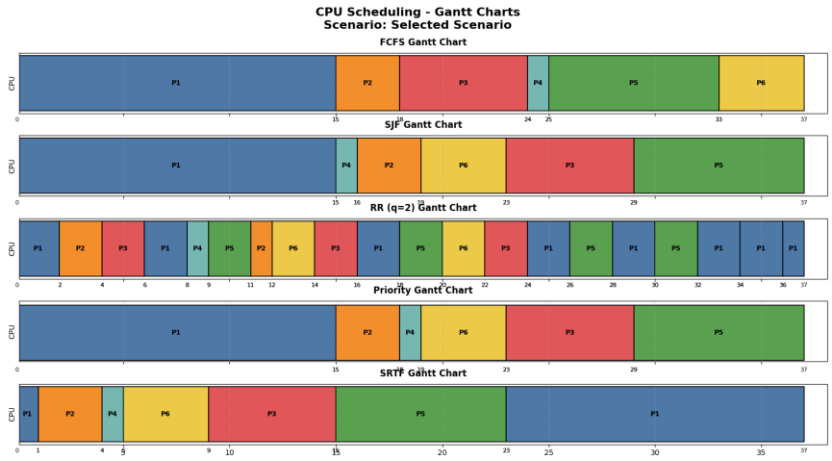


Figure 3: Graphical Representation of Execution Timelines.

Gantt charts tracing the execution timeline, providing visual evidence of scheduling behaviors (e.g., visualizing a convoy effect timeline versus a time-sliced Round Robin timeline).

```
Scenario: Selected Scenario

Best Average Waiting Time    : SRTF (6.83)
Best Average Turnaround Time : SRTF (13.00)
Best Average Response Time   : SRTF (3.17)
Best Throughput             : FCFS (0.16)
```

Figure 4: Evaluating Metrics Dashboard.

A numerical summary dashboard identifying the highest-performing algorithms across key metrics for the simulated workload scenario.

Conclusion & Future Improvements

This project effectively bridged theoretical process scheduling with practical software implementation. By combining rigorous programmatic execution with comprehensive statistical analysis, the simulation clearly demonstrates not only how FCFS, SJF, RR, Priority, and SRTF function, but also where and why each excels or fails under specific workload characteristics. The resulting tool serves as a robust educational resource for analyzing the critical engineering trade-offs inherent in modern operating system design.

Future improvements mapped out for the software include:

Adding support for custom, user-defined process sets via CLI or CSV file input.
Implementing Multilevel Feedback Queue (MLFQ) scheduling for more advanced, enterprise-level simulation.
Developing a web-based or graphical user interface (GUI) front-end for improved interactive exploration.